

DATABROS

AML Challenge Report 2025/26

Gabriele Cabibbo
ID: 2196717

Emanuele Gallo
ID: 2197051

Giorgio Taramanni
ID: 1961217

1 Proposed Method

- **Architecture:** We implemented a bottleneck architecture composed of an encoder, that increases the dimensionality from 1024 to 1408 in a first linear layer, and then decreases it back from 1408 to 1024 to create the bottleneck and moderate overfitting [1] it is followed by a decoder that comprises a layer that increases the dimensionality of the representation from 1024 to 1408, until the output layer projects it to the final output dimension 1536. Each linear layer, made exception for the output layer, is followed by a Layer Normalization and a GELU activation function, and a dropout probability of 0.35 is applied after each nonlinearity.

We selected the hyperparameters with a Grid Search. The eight best-performing models were then ensemble by making the final prediction using a weighted average of the normalized outputs of each single model. Specifically, weights were assigned to each model proportional to the Mean Reciprocal Rank (MRR) obtained by that model during validation.

- **Loss Function:** After multiple submission using a variety of different losses, we decided to use an InfoNCE loss [2][3] since it consistently achieved better results than other objective functions.
- **Training Details:** Before training the model, we split our dataset into a training set containing 90% of the total observation and a validation set containing the remaining data.

We trained the model using the *AdamW* optimizer with mini batches of size 256. During Grid Search, we trained several models, leaving the architecture untouched, looking for the right combination of learning rate and temperature. The weight decay was set to 3×10^{-5} , while other models had learning rates ranging from 1.5×10^{-4} to 1.75×10^{-4} , and temperature between 0.12 and 0.16. The MRR of the best 8 models on our validation set resulted in: 44.53%, 44.52%, 44.49%, 44.47%, 44.46% and, for three models, it was equal to 44.48%.

Finally, our model's prediction is computed as the weighted average of the eight models' normalized outputs with weights proportional to the model's valida-

tion MRR.

2 Results and Discussion

Our final model achieved an MRR of **0.88108** on the public leaderboard and **0.YY** on the private one. The ensembling of these bottleneck models managed to balance the reconstruction capacity of the models that obtained higher performances, with those that performed worse. This probably helped us increase our results on the public leaderboard test set.

3 Conclusion

Once we understood that training our models using a contrastive loss (InfocNCE) improved the models' performances, our architecture choice fell on a quite simple model, also following the results obtained by Bansal et al. [4]. The relationships between the embeddings did not appear too complex, meaning that, for this task, very deep models are not required. A bottleneck model in which information is compressed and then decompressed in steps allows us to extract the most important features of the embeddings. To increase the generalization capabilities of the model and avoid overfitting, we decided to ensemble different models to get more stable predictions.

Although the current ensemble yielded good results, a future implementation would explore an ensembling of more heterogeneous architectures. We observed that models employing a higher hidden dimension achieved better validation performance but exhibited signs of overfitting to the public leaderboard test data. Therefore, the combination of these high capacity models with simpler, more constrained architectures, in a more diverse ensemble could help mitigating overfitting and boost overall performance and stability.

[LINK TO THE REPO](#)

What We Tried

This section details the various approaches we explored during the competition, including those that were not part of our final submission.

Method 1: 4-Layer MLP with MSE

A 4-Layer MLP with sequential linear layers of sizes 1024, 2048, 1536 and 1536 with GELU activation functions (except for the output layer) and Dropout of 0.2 with BatchNorm after each activation function trained with Mean Squared Error (MSE) loss. This was our initial baseline but performed poorly, achieving an MRR of only 0.60931, beating the provided baseline, but hinting that it was overly complex and that the MSE loss wasn't suited for this task.

Method 2: Linear and Orthogonal Mapping

Rodolà et al. [5] suggest that signal encoded in the angles with respect to the anchors is enough to capture the intrinsic shape of the latent space, showing that different latent space that share the same data semantics mostly differ only by an angle preserving transformation. We implemented linear and orthogonal mappings from the text embedding space to the image embedding space, but achieved poor results in terms of MRR.

Method 3: MLP with InfoNCE + Cosine Similarity Loss

We evaluated multiple MLP architectures, training them with a composite loss function that included both InfoNCE and Cosine Similarity terms. Initial experiments demonstrated that the InfoNCE component yielded superior results, leading us to adopt it consistently for all subsequent experiments.

Method 4: Bottlenecks

We combined a convolutional backbone, with 1-D convolutions letting local patterns be mixed before global aggregation, with a bottleneck MLP. Then modified it by inserting a Transformer Bottleneck before the MLP that enriches a compact vector by attending over learnable auxiliary tokens. Another variation of this architecture replaces the convolutional part with a purely linear encoder, inserts the Transformer Bottleneck, and decodes the enriched representation back to the output space. All these architectures aim to compress, refine, and expand features through a structured bottleneck.

Method 5: Flow Matching

The model builds a flow-based mapping from text embeddings to image embeddings, combining a projector, a velocity field with cross-attention, and a contrastive alignment objective. Training mixes flow matching, fast rollout consistency, and InfoNCE contrastive supervision, with EMA stabilization and one-cycle scheduling. Throughout training, we also experimented with different loss-weight configurations, assigning

varying strengths to the flow, rollout, and contrastive components to find the most stable and effective setup [6].

Method 6: Cross-Modal VAE

This model is a cross-modal variational autoencoder (VAE) that compresses text embeddings into a latent space and decodes them into image-embedding space. Training combines KL regularization, reconstruction, triplet, and InfoNCE losses, encouraging both faithful decoding and strong cross-modal alignment. The combination of local (MSE), relational (triplet), and global (InfoNCE) constraints helps the latent space capture shared structure while remaining discriminative.

Method 6: Fine Tuning

For several of the aforementioned architectures, we employed a pre-training strategy utilizing various objective functions, including MSE, InfoNCE loss, and Cosine Embedding loss, followed by fine-tuning of the final layers using a secondary set of losses. This initially improved our models' performances on the public leaderboard, but this improvement faded for models that achieved higher scores without fine tuning.

References

- [1] Kenji Kawaguchi et al. "How Does Information Bottleneck Help Deep Learning?" In: *ICML*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 16049–16096. URL: <https://arxiv.org/abs/2305.18887>.
- [2] Lorenzo Ciampiconi et al. "A survey and taxonomy of loss functions in machine learning." In: *CoRR* (2023). URL: <https://arxiv.org/abs/2301.05579>.
- [3] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. *Representation Learning with Contrastive Predictive Coding*. 2018. URL: <http://arxiv.org/abs/1807.03748>.
- [4] Yamini Bansal, Preetum Nakkiran, and Boaz Barak. "Revisiting Model Stitching to Compare Neural Representations". In: *NeurIPS*. 2021. URL: <https://arxiv.org/abs/2106.07682>.
- [5] Rodolà Emanuele et al. "Latent Space Translation via Semantic Alignment". In: *NeurIPS*. 2023. URL: <https://arxiv.org/abs/2311.00664>.
- [6] Yaron Lipman et al. "Flow Matching for Generative Modeling." In: *ICLR*. 2023. URL: <https://arxiv.org/abs/2210.02747>.